



Engineering Guide

Architecture, Formats, Integration, Security, and Validation

Version 1.0

Last changed: August 7, 2026

Windows • Delphi VCL and FireMonkey • Win32 and Win64

Table of Contents

Table of Contents	i
1. Product Scope and Invariants	1
2. Repository and Build Layout	1
2.1 Toolchain	2
3. System Architecture	2
4. Core Data Model and JSON	3
4.1 Stable keys	3
4.2 Development catalog	3
4.3 Runtime pack	3
5. Project Detection and Scanning	4
5.1 Incremental merge	4
6. Studio UI and Workflow State	4
7. Agent Execution, Provider Settings, and Secret Storage	5
8. Provider HTTP Clients	6
8.1 Reliability and cancellation	6
8.2 Official protocol references	6
9. In-Place AI Translation and Alternatives	7
10. Validation and Runtime Pack Export	8
11. Runtime Engine	8
11.1 Deployment paths	8
12. Integration Planning and Transactions	8
12.1 Generated startup contract	9
13. Self-Localization	9
14. Error Handling and Diagnostics	9
15. Test Strategy and Verified Matrix	10
15.1 Optional live provider testing	10
16. Security Review	11

17. Release and Contribution Workflow.....	11
18. Known Boundaries and Future-Compatible Design.....	12
19. Documentation Generation	12
20. Source References	12

1. Product Scope and Invariants

Delphi App Translation Studio is a Windows-only, open-source localization workspace written in Delphi FireMonkey. It targets Delphi VCL and FMX applications compiled for Win32 and Win64. It is not a runtime translation service and does not require target machines to have Internet access.

The central invariant is separation: scan and catalog operations do not modify target source; provider access exists only in the Studio; integration is explicit, previewed, transactional, backed up, and reversible; target applications consume compact offline JSON only.

- All user interface controls are persisted in `DAT.Studio.MainForm.fmx` and remain editable in the RAD Studio designer.
- No UI is constructed at runtime.
- Stable keys, not source text alone, identify translated properties.
- Original designer text remains the runtime fallback.
- API keys never enter project artifacts.
- Language menu items remain designer-persisted in DFM/FMX resources.

2. Repository and Build Layout

Path	Responsibility
<code>source\core</code>	Catalog types, JSON persistence, project detection, workspace paths, runtime pack generation.
<code>source\agent</code>	Installed Codex/Claude command settings, detection, process execution, logging, and cancellation.
<code>source\scan</code>	VCL/FMX form text, Pascal resourcestrings, extraction rules, diagnostics, incremental merge.
<code>source\validation</code>	Catalog safety and completeness checks.
<code>source\provider</code>	Provider types/settings, Credential Manager, DeepL/Google HTTPS client.
<code>source\runtime</code>	Pack discovery/loading, preference, manager, VCL and FMX applicators.

Path	Responsibility
source\integration	Plan, resource/source rewriting, change sets, transaction, package generation.
source\studio	Designer-authored FMX UI and Studio self-localization.
source\schemas	Development and runtime JSON Schemas.
tools\tests	Foundation, runtime, integration, form-streaming, launch, and self-localization smoke tests.
docs / help / samples	Product documentation, help source, and safe fixtures.

2.1 Toolchain

```
call "C:\Program Files (x86)\Embarcadero\Studio\37.0\bin\rsvars.bat"
msbuild DelphiAppTranslationStudio.dproj /t:Build /p:Config=Debug /p:Platform=Win32
msbuild DelphiAppTranslationStudio.dproj /t:Build /p:Config=Release /p:Platform=Win64
```

The DPROJ identifies the main form as FMX, includes the standard project resource, uses source\provider in its unit search path, and routes EXE/DCU output to bin and dcu by platform/configuration.

3. System Architecture

Boundary	Input	Output
Detection and scan	.dproj/.dpr, text DFM/FMX, Pascal source	TProjectProfile and TProjectScanResult
Catalog	Scan result plus existing development JSON	Merged TTranslationCatalog
Provider	Confirmed source strings plus in-memory credential	Machine-translated entry values
Validation	Development catalog	Errors, warnings, information
Pack builder	Valid catalog	Compact runtime JSON

Boundary	Input	Output
Integration	Project profile, packs, designated menu	Preview package and transactional change set
Runtime	Local JSON and per-user preference	Translated existing controls and locale settings

4. Core Data Model and JSON

DAT.Core.Types defines target framework/platform flags, locale metadata, translation status, catalog entries, and catalog ownership. Development JSON retains source text, translation, stable key, checksum, status, source location, component/property metadata, and locale data. Runtime JSON deliberately omits development-only detail.

4.1 Stable keys

- Form properties use form.component.property, for example frmMain.btnSave.Text.
- resourcestring entries use the Pascal unit and symbol.
- Keys remain stable when wording changes, allowing source-changed detection.
- Fallback source text remains in the compiled DFM/FMX or code.

4.2 Development catalog

```
{
  "schemaVersion": 3,
  "applicationId": "SampleApp",
  "sourceLanguage": "en-US",
  "locale": {"languageCode": "it-IT", "nativeLanguageName": "Italiano"},
  "entries": [{"key": "frmMain.btnSave.Text", "sourceText": "Save",
    "translatedText": "Salva", "status": "aiDraft",
    "translationOrigin": "codex",
    "translationConfidence": "high",
    "runtimeApplication": "automatic"}]
}
```

4.3 Runtime pack

```
{
  "schemaVersion": 1,
  "applicationId": "SampleApp",
  "languageCode": "it-IT",
  "nativeLanguageName": "Italiano",
  "sourceCatalogChecksum": "...",
  "strings": {"frmMain.btnSave.Text": "Salva"}
}
```

The schema files are `source\schemas\development-project.schema.json` and `runtime-language-pack.schema.json`. Changes require schema-version handling, round-trip fixtures, and compatibility notes.

5. Project Detection and Scanning

`DAT.Core.ProjectDetection` reads project metadata, resolves the DPR, detects VCL or FMX through project/form evidence, records Win32/Win64 support, and enumerates forms and Pascal sources.

`DAT.Scan.Project` coordinates form and resourcestring scanners while measuring elapsed milliseconds.

`DAT.Scan.FormText` parses text DFM/FMX without instantiating target forms. `DAT.Scan.PascalResources` extracts resourcestring declarations. `DAT.Scan.TextCodec` preserves Delphi text encodings and escaped string syntax. `DAT.Scan.Rules` centralizes designated property decisions.

5.1 Incremental merge

1. Index the existing catalog by stable key.
2. Add unseen keys as needs-translation.
3. Preserve translation and status when source text is unchanged.
4. Preserve existing translation but mark source-changed when source text differs.
5. Mark catalog-only entries obsolete unless excluded.
6. Report new, changed, unchanged, and obsolete counts.

6. Studio UI and Workflow State

`DAT.Studio.MainForm.fmx` contains the complete orange-and-blue interface. The form persists `WindowState=wsMaximized`, uses client-aligned workflow cards, anchors resizable work areas to every relevant edge, and keeps the status card at the bottom. The workflow selection rectangle moves among Project, Scan, Translate, Validation, Export, Integration, and Engine Settings. Language and engine choices are designer-owned combo boxes. Translate Automatically, Cancel Translation, Prompt, Reload, agent configuration controls, and the monitor timer are persisted designer objects.

`DAT.Studio.MainForm.pas` contains event and state logic only; it does not construct controls.

Each workflow `TLabel` explicitly persists `HitTest=True`. FireMonkey labels default to `HitTest=False`, which would otherwise pass mouse events through the visible label even when an `OnClick` event is assigned. The setting remains editable in the Object Inspector.

The form owns the project profile, scan result, catalog, validation result, integration change set, provider settings, and per-provider session-key strings. Destructors release owned objects. Catalog updates invalidate validation and export state.

FMX validation requirement. A successful compile is insufficient. Persisted FMX property-type errors surface only while streaming. StudioFormSmokeTests directly constructs the form, and launch tests verify the real title in every configuration.

7. Agent Execution, Provider Settings, and Secret Storage

DAT.Agent.Execution persists only the selected command-line engine, executable path, and model choice. It detects supported installations, starts the agent without a console window, passes the translation contract through inherited standard input, redirects stdout/stderr to a project-local diagnostic log, polls the process from the designer-owned timer, and supports controlled cancellation. Codex runs non-interactively with workspace-write sandboxing in the target project; Claude runs in print mode with streamed JSON and accepted workspace edits. Authentication remains owned by the installed vendor CLI.

DAT.Provider.Settings persists only provider, DeepL plan, remember choice, timeout, and batch size in %LOCALAPPDATA%\DelphiAppTranslationStudio\provider-settings.json. Bounds are normalized to 5-300 seconds and 1-50 strings.

DAT.Provider.CredentialStore uses CredWriteW, CredReadW, CredDeleteW, and CredFree with CRED_TYPE_GENERIC and local-machine persistence. Credential targets are provider-specific and begin TMartinDub/DelphiAppTranslationStudio/. Secret bytes are cleared after conversion where practical.

Choice	Persistence	Replacement behavior
Remember checked	Windows Credential Manager	Writes/replaces selected provider credential and clears session copy.
Remember cleared	Process memory only	Deletes selected provider stored credential and retains entered key until shutdown.
Remove Key	None	Deletes stored and session copies for selected provider.

- The masked edit never displays a stored credential.
- The effective key resolves new field text, then session memory, then Credential Manager.
- No error message contains the key, request headers, or provider response body.
- Credential operations occur only when needed; the form can start without network access.

8. Provider HTTP Clients

DAT.Provider.Client uses Delphi THTTPClient. It validates a nonblank key, normalizes settings, builds provider-specific JSON, sends HTTPS POST requests, parses provider response arrays, verifies result counts, and returns translations in source order.

Provider	Endpoint	Authentication	Payload
DeepL API Free	https://api-free.deepl.com/v2/translate	Authorization: DeepL-Auth-Key	text array, source_lang, target_lang
DeepL API Pro	https://api.deepl.com/v2/translate	Authorization: DeepL-Auth-Key	text array, source_lang, target_lang
Google Basic v2	https://translation.googleapis.com/language/translate/v2	X-Goog-API-Key	q array, source, target, format=text

8.1 Reliability and cancellation

- Batch size is capped at 50 and Google language tags are reduced to base language where appropriate.
- DeepL source codes are normalized to two-letter uppercase; targets retain supported regional forms.
- HTTP 429 and 5xx responses receive three bounded attempts with short backoff.
- Other non-2xx statuses fail immediately with provider, status, and corrective categories.
- A cancellation callback is checked between batches.
- Test Connection translates one harmless English phrase to Italian.

8.2 Official protocol references

- DeepL authentication: <https://developers.deepl.com/docs/getting-started/auth>
- DeepL quickstart: <https://developers.deepl.com/docs/getting-started/quickstart>
- Google Basic translate method:
<https://docs.cloud.google.com/translate/docs/reference/rest/v2/translate>
- Google Cloud Translation authentication:
<https://docs.cloud.google.com/translate/docs/authentication>

- Google API-key best practices: <https://docs.cloud.google.com/docs/authentication/api-keys-best-practices>

9. In-Place AI Translation and Alternatives

The canonical workflow combines `DAT.Core.AITranslation` with `DAT.Agent.Execution`. `Translate` Automatically saves an exact recovery snapshot, ensures the terminology profile, writes the agent contract, records the file SHA-256, locks Studio editing, launches the selected signed-in command-line agent, supplies the complete task through standard input, and monitors both the child process and catalog from a designer-owned timer.

A zero process exit triggers protected reload and automatic save. A nonzero exit or developer cancellation restores the snapshot. Prompt and Reload remain explicit recovery controls for diagnosis or manually continued work; they are not required during the normal one-button path.

Reload parses the external file and compares application, framework, language, locale formats, entry count/order, keys, source/checksums, context, developer notes, runtime classification, and wiring confirmation with the pre-session snapshot. Only Needs Translation, Source Changed, Error, and existing AI Draft entries are eligible. Machine-translated, Imported, Edited, Reviewed, Approved, Excluded, and Obsolete entries are protected. Any protected mutation rejects the reload.

Allowed changes are normalized to AI Draft while Codex/Claude origin, high/medium/low confidence, and review notes are retained. A metadata-only confirmation is retained when an existing translation remains correct after a source change.

CSV/JSON interchange, manual editing, and direct Google/DeepL translation remain alternatives. Provider output receives Machine translated status with its provider origin.

The Studio never copies a translation from one stable key to another automatically. Exact-source matches are suggestions only. Explicit acceptance creates an Edited result and does not inherit Reviewed or Approved status.

- Mark Reviewed requires nonblank translated text.
- Approve requires the entry to have reached Reviewed first.
- Translation origin is independent of linguistic status.
- Low confidence, explicit AI notes, inconsistent repeated terms, and structural defects form the focused exception queue; a valid AI draft is not itself an exception.
- Structural validity, linguistic status, automatic runtime coverage, and manual wiring readiness are separate measures.
- A Pascal resourcestring remains manual wiring until the developer confirms the generated `TranslateText` call site.

10. Validation and Runtime Pack Export

DAT.Validation.Catalog validates metadata, missing target text, duplicates, changed source, Delphi indexed and sequential Format arguments, accelerators, low AI confidence, explicit AI review notes, inconsistent repeated-source terminology, and status conditions. Export is blocked by errors. Manual resourcestring wiring is reported separately from structural errors. DAT.Core.RuntimePack serializes only runtime-required metadata and strings and records a source-catalog checksum.

Locale data is retained in the runtime pack so DAT.Runtime.Manager can expose a pack-specific TFormatSettings without globally mutating the developer's source code.

11. Runtime Engine

DAT.Runtime.LanguagePack loads and discovers JSON packs, checks application identity, and provides fallback lookup. DAT.Runtime.Preference reads/writes the selected locale. DAT.Runtime.Manager owns the active pack, discovery, preference, translation lookup, and locale format settings.

DAT.Runtime.VCL and DAT.Runtime.FMX traverse existing component trees and supported collection properties. They apply values by stable key to already designer-created controls. Missing keys retain source text. The adapters do not create controls or rearrange layouts.

11.1 Deployment paths

- Packs: <ExecutableDirectory>\Localization\Languages\<locale>.json.
- Preference: %LOCALAPPDATA%\<ApplicationId>\language.ini.
- Studio preference: %LOCALAPPDATA%\DelphiAppTranslationStudio\language.ini.
- Developer catalogs remain in the target source tree under Localization\Development.

12. Integration Planning and Transactions

DAT.Integration.Plan builds a human-readable plan. DAT.Integration.Package creates the generated application unit, framework adapter, shared runtime units, JSON packs, language-menu manifest, and deployment script. DAT.Integration.Engine builds every proposed file change in memory.

TIntegrationFileChange.ExactReviewText produces a complete line-numbered original/proposed LCS diff; generated files are shown in full.

The Studio records each selected preview file as viewed. The final review checkbox remains disabled until every file has been viewed, and Apply remains disabled until the checkbox is explicitly checked.

DAT.Integration.Transaction then checks source state, creates a verified pre-change backup, writes files atomically, rolls back failures, and records a manifest for restore.

DAT.Integration.MenuResource modifies text DFM/FMX menus idempotently and preserves designer editability. DAT.Integration.DelphiSource applies narrowly scoped DPR, PAS, and DPROJ wiring. For FMX, it preserves an existing root OnCreate handler or persists datTranslationFormCreate in the FMX and adds ApplyTranslation(Self) to its Pascal method. This respects the FMX form streaming lifecycle and avoids translating the first form from the DPR immediately after CreateForm.

12.1 Generated startup contract

- InitializeTranslation runs before form creation.
- VCL applies the first created form through the DPR startup wiring.
- FMX applies each form in its designer-persisted OnCreate handler after streaming is complete.
- SelectLanguageMenuItem derives a locale from datLanguage_<locale> component names.
- TranslateText provides resourcestring/code fallback access.
- Runtime ownership is finalized safely.

Original code preservation. Integration adds narrowly scoped calls and units but does not replace original captions or rewrite every control. The original DFM/FMX values continue to be the source-language fallback.

13. Self-Localization

DAT.Studio.Translation resolves the project/deployed pack directory, initializes a Studio runtime before form creation, applies the active pack in FormCreate, and maps persisted language-menu names to locale codes. Self-integration recognizes the existing runtime and plans only the menu-resource change, preventing duplicate startup wiring.

14. Error Handling and Diagnostics

- Project, scan, catalog, validation, export, integration, credential, and provider errors are caught at Studio event boundaries and summarized in the status card.
- Provider exceptions carry an optional HTTP status but not response bodies.
- Form scanners return file/line diagnostics without instantiating target UI.
- Integration previews show a complete line-numbered exact diff for every affected file before mutation.
- Transaction errors trigger rollback and retain backup evidence.
- Provider keys must never be added to troubleshooting logs.

15. Test Strategy and Verified Matrix

Test	Coverage
FoundationSmokeTests	Detection, scan-to-catalog, protected in-place AI session/reload, schema/provenance round-trip, independent CSV fallback, review/approval, validation, runtime pack, preference, exact diff, integration, self change set.
VCLRuntimeSmokeTests	VCL controls, menu items, locale, generated unit.
FMXRuntimeSmokeTests	All representative FMX form properties, menu items, locale, generated unit.
StudioFormSmokeTests	Direct FMX stream/create, maximized client-aligned pages, designer-owned agent controls/timer, exact-review controls, linguistic action wiring, and provider alternative activation.
RunRuntimeSmokeTests.ps1	Both compilers, protected in-place AI pilot pipeline, disposable integrated VCL/FMX builds, deployed Italian pack, and required Italian launch title.
RunStudioLaunchSmokeTests.ps1	Debug/Release Win32/Win64 real main-window title.
RunStudioSelfLocalizationSmokeTest.ps1	Italian Studio title in all four configurations with state restoration.

On August 7, 2026, the release gate passed with real compilable VCL and FMX samples under both Win32 and Win64. Each sample was scanned, translated by a deterministic Codex-style direct edit of its canonical JSON, protected-field reviewed, reloaded as AI Draft with provenance, reviewed, approved, validated, exported, integrated, built, deployed, and launched with the required Italian title. Snapshot creation and protected mutation rejection passed. The Studio also built with zero warnings and errors in Debug and Release for both architectures, streamed its FMX form directly, launched normally, and self-localized to Italian in all four configurations.

15.1 Optional live provider testing

Automated fixtures must not contain live keys. Optional provider acceptance uses an owner-supplied restricted key through Engine Settings, confirms Test Connection, translates a small disposable catalog,

checks Machine translated status and provider provenance, then removes or rotates the test key. Provider availability is external state and is not a release blocker for the primary Codex/Claude workflow or offline runtime.

16. Security Review

Threat	Control
Key committed to Git	No key field in settings JSON/catalogs; Credential Manager or memory only; release secret scan.
Key leaked through URL/log	Authentication headers, redacted errors, no body/header logging.
Wrong target changed	Profile display, previewed file change set, explicit Apply.
Unrecoverable source mutation	Verified G-drive project backup by workflow policy plus transaction backup/manifest/restore.
Installed-folder write failure	Per-user language preference; packs are read-only deployment assets.
Machine mistranslation shipped	Machine-translated status, validation, required human review.
Agent corrupts catalog structure	Exact pre-session snapshot, protected-field comparison, all-or-nothing reload, and explicit recovery.
Pack for wrong application	applicationId validation in pack discovery/loading.

Windows Credential Manager follows Microsoft guidance for application credentials (CredWrite/CredRead). It protects persistence under the signed-in Windows context; it does not make a key safe from malware running as that user. Provider-side restrictions, quotas, rotation, and least privilege remain necessary.

17. Release and Contribution Workflow

1. Read global and repository directives.
2. Confirm the exact requested scope and make a pre-change backup when changes are approved.

3. Keep UI edits in the FMX designer resource and source event logic separate.
4. Add deterministic tests without credentials.
5. Run Win32/Win64 Debug/Release builds elevated.
6. Run runtime, form-streaming, launch, and self-localization suites.
7. Update phase notes, help, User Guide, Engineering Guide, real TOCs, PDFs, and visual QA.
8. Check for a stale zero-byte .git\index.lock with no Git process.
9. Review status, stage only intended files, commit clearly, and push every configured remote.

18. Known Boundaries and Future-Compatible Design

- Only Windows Win32/Win64 Delphi applications are supported.
- Google Advanced v3, OAuth/service-account workflows, and other providers are not implemented.
- Provider operations are optional explicit Studio actions; target runtime remains offline.
- The Studio launches a separately installed Codex CLI or Claude Code command but never impersonates, embeds, or authenticates that service. The developer installs and signs in to the selected CLI using vendor-supported instructions; the Studio stores only non-secret command settings.
- No automatic control resizing/reflow is performed.
- Binary DFM conversion is outside the scanner.
- The generated menu selection persists the locale; restarting is the conservative way to ensure every form and application-specific string is refreshed.
- Provider account terms and language support are external and must be rechecked for each release.

19. Documentation Generation

The editable guides are generated from actual source and engineering notes with python-docx. Each DOCX contains a Word TOC field, a dedicated TOC section, and a new-page content section. Microsoft Word COM updates the TOC and fields, repaginates, saves the DOCX, and exports the companion PDF through ExportAsFixedFormat. PDFs are rendered to page images and inspected before release.

20. Source References

- Repository source, forms, project metadata, schemas, and smoke tests as of August 7, 2026.
- Microsoft credential handling: <https://learn.microsoft.com/en-us/windows/win32/secbp/handling-passwords>

- Windows CREDENTIAL structure: <https://learn.microsoft.com/en-us/windows/win32/api/wincred/ns-wincred-credentialw>
- DeepL developer documentation: <https://developers.deepl.com/docs/getting-started/auth>
- Google Cloud Translation documentation: <https://docs.cloud.google.com/translate/docs/authentication>
- Google API-key guidance: <https://docs.cloud.google.com/docs/authentication/api-keys-best-practices>
- OpenAI Codex CLI reference: <https://developers.openai.com/codex/cli/reference/>
- Anthropic Claude Code CLI reference: <https://docs.anthropic.com/en/docs/claude-code/cli-usage>